COMPANION TO THE PROJECT PLANS

Tools & Concepts, Explained from Zero

A no-experience-needed guide to Python, Jupyter, Git & how the simulation actually works

Read this before Week 1 · no jargon left behind

Who this is for: You're comfortable with engineering and maths but have written little or no code, and the words "Python", "Jupyter" and "GitHub" feel like a wall.
The promise: By the end you'll know what each tool is, why it's in the plan, what you genuinely need on day one, and a gentle warm-up path that gets you coding-confident before the rocket project formally begins.

00 · Read this first: the right mindset

You are not becoming a software developer. You are learning to **follow engineering recipes** and adjust them. For this project, coding is plumbing — a way to make a calculator that repeats a small sum thousands of times so you don't have to. That's genuinely all a trajectory simulation is.

> **THE SINGLE MOST REASSURING FACT**
>
> The hard part of your project is the *engineering thinking* — the physics, the assumptions, the trade-offs — and you already have that from your BEng. The code is just the tool that does the arithmetic. Plenty of excellent engineers write modest, functional code. Your report is marked on reasoning and clarity, **not** on elegant programming.

- **You will see error messages constantly.** That is normal and not a sign you're failing — even professionals see them all day. An error is just the computer pointing at a line and saying "I didn't understand this bit."
- **You don't need to memorise anything.** Copy working examples, change the numbers, see what happens. That's how everyone learns.
- **You can build the whole rocket project with about six concepts.** They're all in Section 4.

01 · The map: what each tool is & whether you need it on day one

Here's everything named in the plans, in one line each, ranked by when you actually need it.

Tool / term	What it is, in one sentence	Need it when?
Python	The language you write your instructions in — like the "calculator" that runs your simulation.	Day one (rocket).
Anaconda	A single free installer that puts Python + Jupyter + the maths/plotting libraries on your computer in one go.	Day one — install this and you're done setting up.
Jupyter (Notebook)	A friendly page where you write code in small chunks and see results & graphs right underneath — ideal for beginners.	Day one — this is where you'll work.
NumPy	A maths toolkit add-on for Python (arrays, square roots, etc.). You just `import` it and use it.	Week 2.
Matplotlib	The add-on that draws your graphs (altitude vs time, etc.).	Week 2.
Git	A "save-points" system that tracks every version of your work so you can never lose it or break it permanently.	Optional — can add at the end.
GitHub	A website that stores your Git project online — doubles as a shareable portfolio link for applications.	Optional — nice for your CV.
Spreadsheet (CubeSat)	Excel / Google Sheets — for the power, mass & link budget tables. No coding at all.	CubeSat weeks.
CAD (CubeSat)	3-D modelling software (Fusion 360 / Onshape) to draw the satellite. Visual, not code.	CubeSat week 3.

THE HONEST SHORTLIST

To start the rocket project you only need **two things installed**: **Anaconda** (which includes Python, Jupyter, NumPy and Matplotlib) and a web browser. Git/GitHub are a bonus you can bolt on at the very end. Don't let them intimidate you out of the gate.

02 · Python & Jupyter: your workbench

What is Python, really?

Python is just a way of writing instructions that a computer follows top to bottom. If you can write a recipe — "take this number, multiply it by that, write down the answer" — you can write Python. It reads almost like English:

```
# a line starting with # is a note to yourself — the computer ignores it
mass = 20            # kilograms
gravity = 9.81       # m/s^2
weight = mass * gravity
print(weight)        # shows 196.2 on screen
```

That's real, complete Python. `*` means multiply, `print()` means "show me the answer." Nothing more mysterious than that.

What is Jupyter, and why use it?

A **Jupyter Notebook** is a page in your web browser split into **cells** — little boxes. You type a chunk of code into a cell, press `Shift+Enter`, and the answer (or graph) appears directly beneath it. It's perfect for beginners because you build and test in small, visible steps instead of running a whole program blind.

WHY BEGINNERS LOVE IT

- See results and plots **inline**, instantly.
- Run one small piece at a time; fix mistakes locally.
- Mix notes, maths and code on one page — great for your own understanding.

SCRIPT VS NOTEBOOK (DON'T OVERTHINK)

A "script" (a `.py` file) is the whole program in one file, run start-to-finish. A notebook is interactive and chunked. **Learn and explore in a notebook**; later you can copy your working code into a tidy `.py` file for the final submission — the plan's `src/` folder. Either is fine for marks.

YOUR ONE-TIME SETUP (≈ 20 MINUTES)

1. Download **Anaconda Distribution** from anaconda.com (free, individual edition). This bundles Python, Jupyter, NumPy and Matplotlib so you never touch a command line to install anything.
2. Open **Anaconda Navigator**, then launch **JupyterLab** (or "Jupyter Notebook").
3. Click "New → Python 3 Notebook." Type `print("hello rocket")` in the first cell and press `Shift+Enter`.
4. If you see `hello rocket` appear — congratulations, your entire environment works. That's the hardest setup step done.

03 · The six concepts that build the whole project

Everything in the rocket simulation is made from these. Each one in plain English with a tiny rocket-flavoured example.

1 · VARIABLE — A LABELLED BOX THAT HOLDS A VALUE

```
thrust = 1500     # newtons — store a number under a name
burn_time = 6     # seconds
```

Later you just write `thrust` instead of `1500`. Change it in one place and everything updates.

2 · LIST / ARRAY — A ROW OF VALUES

```
altitudes = [0, 12, 47, 105]   # a list of heights over time
```

You'll collect altitude at each instant into one of these, then hand the whole row to the plotter. (NumPy gives a faster, maths-friendly version called an array — same idea.)

3 · FUNCTION — A REUSABLE MINI-MACHINE: FEED IT INPUTS, GET AN OUTPUT

```
def drag(density, velocity, cd, area):
    return 0.5 * density * velocity * abs(velocity) * cd * area
```

Now `drag(1.225, 100, 0.5, 0.0079)` gives the drag force. Write the formula once, reuse it forever. This is exactly how the plan's `deriv()` function works.

4 · LOOP — "DO THIS AGAIN AND AGAIN"

```
for step in range(1000):
    # update velocity and altitude by one tiny time-step
    # ...repeat 1000 times
```

This is the heart of the simulation: take one tiny step forward in time, recompute the forces, repeat. The computer does the boring repetition for you.

5 · IF-STATEMENT — MAKE A DECISION

```
if time < burn_time:
    thrust = 1500      # engine on
else:
    thrust = 0         # engine off — coasting
```

This is precisely how your code switches between the **powered** and **coast** phases the brief asks for.

6 · IMPORT — BORROW A READY-MADE TOOLKIT

```
import numpy as np                # maths toolkit, nicknamed np
import matplotlib.pyplot as plt   # plotting toolkit, nicknamed plt
plt.plot(times, altitudes)          # draw the graph
```

You don't write the graph-drawing code yourself — you import Matplotlib and ask it nicely.

> **THAT'S THE WHOLE TOOLBOX**

> Variable, array, function, loop, if-statement, import. Re-read those six and you can read — and write — every line in the rocket plan. There is nothing else hiding.

04 · NumPy & Matplotlib: the two add-ons you'll lean on

NUMPY = THE MATHS HELPER

Gives you square roots, exponentials, and "arrays" (rows of numbers you can do maths on all at once). Whenever the plan writes `np.array(...)` or `np.exp(...)`, that's NumPy. You just import it and call it.

MATPLOTLIB = THE GRAPH MAKER

Turns your rows of numbers into the altitude/velocity/acceleration plots the report needs. Three lines gets you a labelled graph:

```
plt.plot(times, altitudes)
plt.xlabel("Time (s)")
plt.ylabel("Altitude (m)")
```

05 · How the simulation actually works (no code)

Forget code for a moment. Here's the whole idea in plain physics, the way you'd explain it to a colleague.

You can't solve the rocket's motion in one shot because the forces keep changing — fuel burns off, drag depends on speed, air thins with altitude. So instead you **chop time into tiny slices** (say 0.01 s each) and, for each slice, pretend the forces are constant for that brief instant. Then:

1. **Look at "now":** current altitude, velocity and mass.
2. **Add up the forces:** thrust (if still burning) − weight − drag.
3. **Get acceleration:** force ÷ mass (Newton's 2nd law).
4. **Step forward a tiny bit:** new velocity = old + acceleration × Δt; new altitude = old + velocity × Δt.
5. **Repeat** thousands of times until the rocket stops climbing (velocity hits zero = apogee).

It's a flip-book: each frame is a tiny step, and playing them in sequence is the trajectory. That's the **loop** from Section 3 doing its job.

> **SO WHAT ARE "EULER" AND "RK4"?**
> They're just two recipes for *how* to take that tiny step. **Euler** is the simplest (the "new = old + rate × Δt" above) — easy to understand, slightly inaccurate. **RK4** is a smarter step that peeks ahead a few times within each slice to take a better-aimed step, so it's much more accurate for the same slice size. You don't need to derive RK4 — treat it as a tested recipe you copy from the plan and trust. Start with Euler to understand the idea, then swap in RK4.

Decoding the scary-looking snippet from the plan

Here's the `deriv()` function from the rocket plan, with every line translated:

```
def deriv(state, t):           # a machine: give it the current situation
    h, v = state                # unpack current height and velocity
    m = mass(t)                 # how much the rocket weighs right now
    thrust = T if t < t_b else 0   # engine on during burn, else off
    drag = 0.5*rho(h)*v*abs(v)*Cd*A    # the drag force formula
    a = (thrust - m*g - drag)/m # Newton's 2nd law → acceleration
    return np.array([v, a])      # hand back the rates of change
```

Read top-to-bottom, it's just steps 1–3 of the flip-book above, written down. Nothing in it is beyond your physics.

06 · Git & GitHub, demystified (and why you can relax about them)

Git is a system of *save points*. Every time you reach a working state, you "commit" — Git remembers that exact version forever. If you break something tomorrow, you can roll back to any earlier save. It's an undo button that never forgets.

GitHub is a website that stores those save points *online*. Two benefits: a cloud backup so your laptop dying can't lose your project, and a public link you can put on your CV/personal statement so admissions tutors can click through and see real work.

> **YOU DO NOT NEED THIS TO START — PROMISE**
>
> You can build the entire rocket project with zero Git, just saving files normally. Git/GitHub is a finishing touch that makes your work shareable and impressive. If it's adding stress, skip it until Week 6 and add it then. The project's marks don't depend on it.

THE LEAST-PAINFUL WAY TO USE IT (NO COMMAND LINE)

1. Install **GitHub Desktop** (desktop.github.com) — a clickable app, no typing commands.
2. Make a free account at github.com; create a new repository called `rocket-sim`.
3. Put your project folder in it. Each time you finish something, type a short message ("added drag") and click **Commit**, then **Push** (sends it online).
4. That's it. Your shareable link is `github.com/yourname/rocket-sim`.

If even that feels like too much at first, an honest fallback for backup is: keep your project in a cloud-synced folder (OneDrive/Google Drive/Dropbox) and zip a dated copy at each milestone. Add proper Git later.

07 · What to do when you get stuck (you will, and it's fine)

- **Read the last line of the error first.** Python puts the actual problem at the bottom, often naming the exact line. `NameError` = you used a name you didn't define; `SyntaxError` = a typo like a missing bracket or colon; `TypeError` = you mixed incompatible things (e.g. text and a number).
- **Change one thing at a time** and re-run. Tiny experiments beat big rewrites.
- **Paste the error into a search engine or an AI assistant** and ask "what does this mean and how do I fix it?" This is a normal, legitimate part of how everyone codes.
- **Print things out.** Add `print(velocity)` inside your loop to see what's actually happening — the simplest debugging tool there is.

08 · Your gentle "Week 0" warm-up (≈ 4–6 hours, before Week 1)

Because you're starting from little code experience, spend a few short sessions here *before* the rocket plan's Week 1, so the real project feels like applying skills rather than learning them cold.

1. **Install Anaconda & run "hello rocket"** (Section 2 setup box). ~30 min.

2. **Work through the first ~5 short chapters of *Automate the Boring Stuff with Python*** (free online): variables, maths, lists, functions, loops, if-statements. Stop at file/web stuff — you don't need it. ~3 hrs.

3. **Make one graph yourself:** in a notebook, create two lists (e.g. seconds `[0,1,2,3]` and heights `[0,5,18,40]`) and plot them with Matplotlib. Seeing your own graph appear is the confidence unlock. ~1 hr.

4. **Write a 10-line "falling object" loop:** drop a ball, update velocity and position each step under gravity, print the values. This *is* a baby trajectory simulation — Week 2 is just this plus thrust and drag. ~1 hr.

IF YOU DO ONLY ONE THING

Do step 4. A loop that steps an object through time under gravity is the entire skeleton of the rocket project. Once that clicks, everything in the plan is an extension of it.

09 · Free beginner resources

Resource	What it's good for
Automate the Boring Stuff with Python (free to read online)	The friendliest no-experience intro to the six core concepts. Read the early chapters only.
Python.org "Beginner's Guide"	Official starting point; reassuringly simple first steps.
Matplotlib "Pyplot tutorial" & NumPy "Absolute beginners" guide	Short official walk-throughs for plotting and arrays — skim when Week 2/3 needs them.
GitHub Desktop docs / GitHub "Hello World" guide	Click-based Git with no command line — for the optional Week 6 polish.
An AI assistant	Paste any error or ask "explain this line." Great for unblocking — just make sure you understand the answer, since your report must be your own reasoning.

10 · Mini-glossary

Term	Plain meaning
Script / `.py` file	A whole program saved in one file, run start to finish.
Cell	One chunk of code in a Jupyter notebook that you run on its own.
Library / package / module	A bundle of ready-made code you `import` and use (NumPy, Matplotlib).
Argument / parameter	The inputs you hand to a function (e.g. the velocity you give `drag()`).
Return	The answer a function hands back to you.
Integer / float	A whole number (6) vs a decimal number (9.81).
String	Text, written in quotes, like `"Time (s)"`.
Commit / push (Git)	Save a version locally / send it up to GitHub.
Repository ("repo")	The project folder Git is tracking.
Δt (time-step)	The tiny time slice the simulation jumps forward by each loop.
Euler / RK4	Two recipes for taking one time-step; RK4 is more accurate.

FINAL WORD

None of this is beyond you. You're an engineer learning a tool, not a beginner learning engineering. Install Anaconda, do the Week 0 falling-object loop, and the rocket plan will read like a series of small, sensible extensions — each one a step you already understand. Start small, run things often, and let the errors teach you.